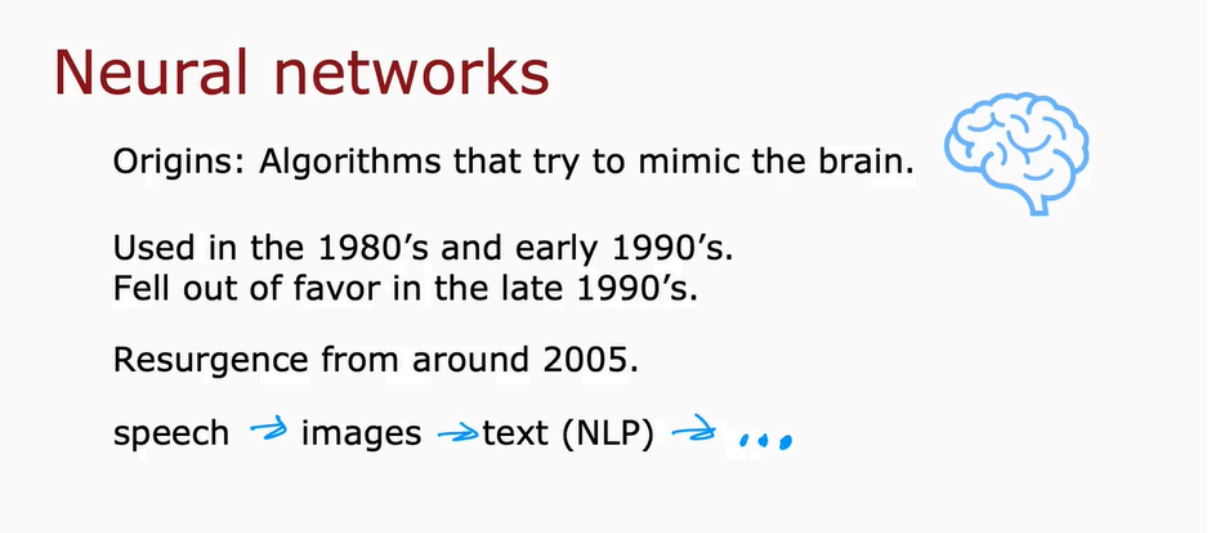
Neural Networks intuition

Neural Networks, also called deep learning algorithms, as well as decision trees.

Neurons and the brain

When neural networks were first invented many decades ago, the original motivation was to write software that could mimic how the human brain or how the biological brain learns and thinks.

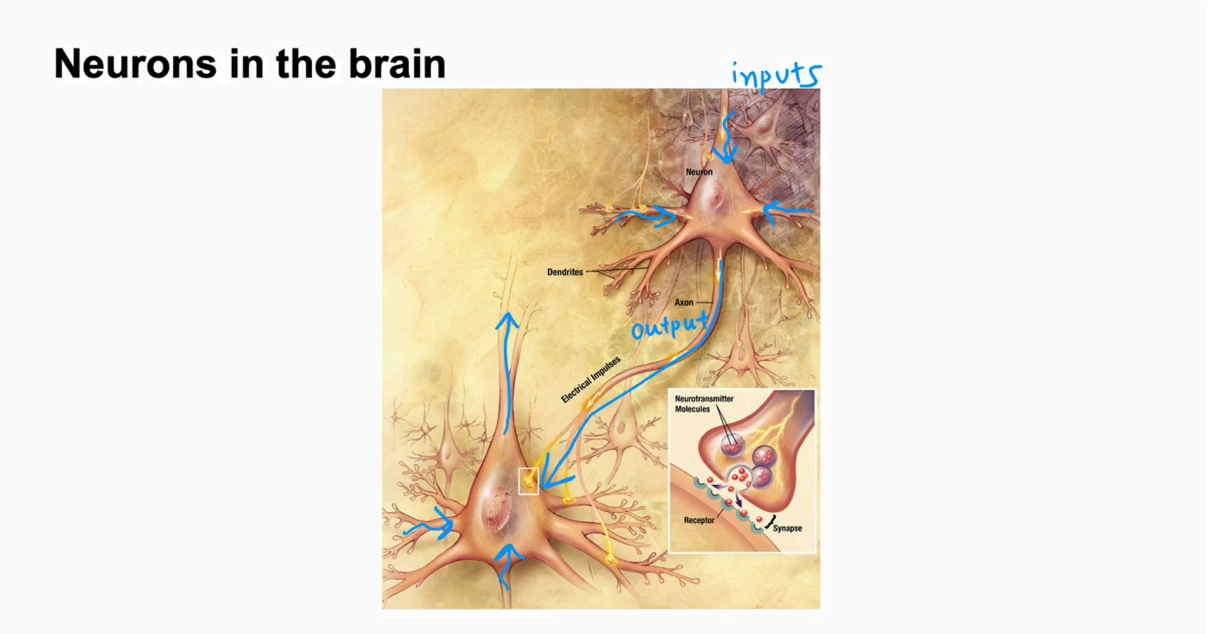
Even though today, neural networks, sometimes also called artificial neural networks, have become very different than how any of us might think about how the brain actually works and learns. Some of the biological motivations still remain in the way we think about artificial neural networks or computer neural networks today.



- Let's start by taking a look at how the brain works and how that relates to neural networks. The human brain, or maybe more generally, the biological brain demonstrates a higher level or more capable level of intelligence and anything else would be on the bill so far.
- So neural networks has started with the motivation of trying to build software to mimic the brain. Work in neural networks had started back in the 1950s, and then it fell out of favor for a while. Then in the 1980s and early 1990s, they gained in popularity again and showed tremendous

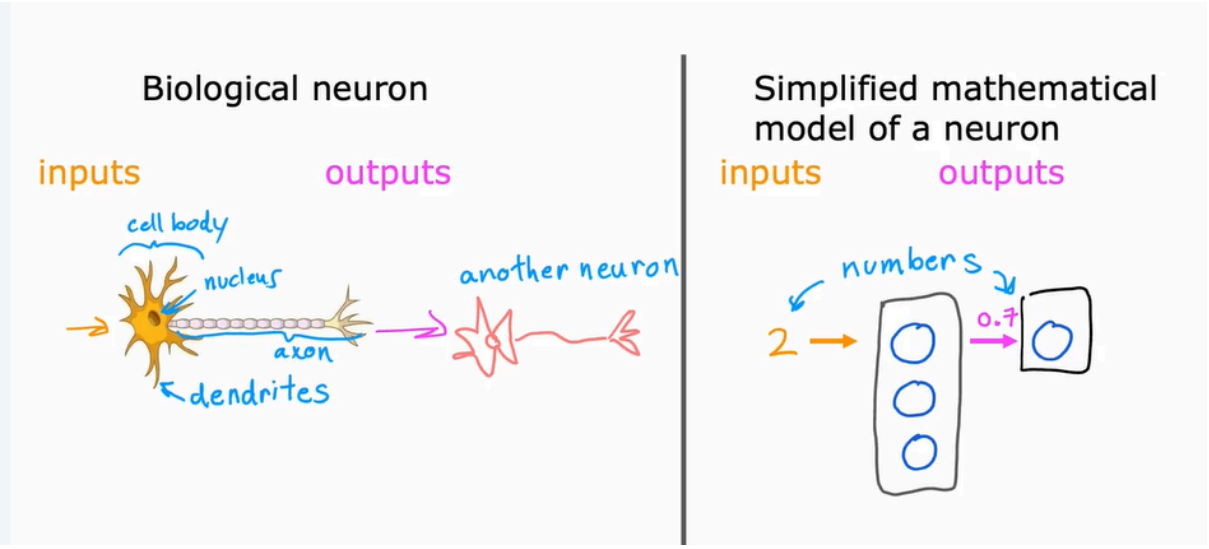
traction in some applications like handwritten digit recognition, which were used even backed then to read postal codes for writing mail and for reading dollar figures in handwritten checks.

- But then it fell out of favor again in the late 1990s. It was from about 2005 that it enjoyed a resurgence and also became re-branded little bit with deep learning. One of the things that surprised me back then was deep learning and neural networks meant very similar things. But maybe under-appreciated at the time that the term deep learning, just sounds much better because it's deep and this learning. So that turned out to be the brand that took off in the last decade or decade and a half. Since then, neural networks have revolutionized application area after application area.
- I think the first application area that modern neural networks or deep learning, had a huge impact on was probably speech recognition, where we started to see much better speech recognition systems due to modern deep learning and authors such as [inaudible] and Geoff Hinton were instrumental to this, and then it started to make inroads into computer vision. Sometimes people still speak of the ImageNet moments in 2012, and that was maybe a bigger splash where then [inaudible] draw their imagination and had a big impact on computer vision. Then the next few years, it made us inroads into texts or into natural language processing, and so on and so forth. Now, neural networks are used in everything from climate change to medical imaging to online advertising to prouduct recommendations and really lots of application areas of machine learning now use neural networks. Even though today's neural networks have almost nothing to do with how the brain learns, there was the early motivation of trying to build software to mimic the brain.



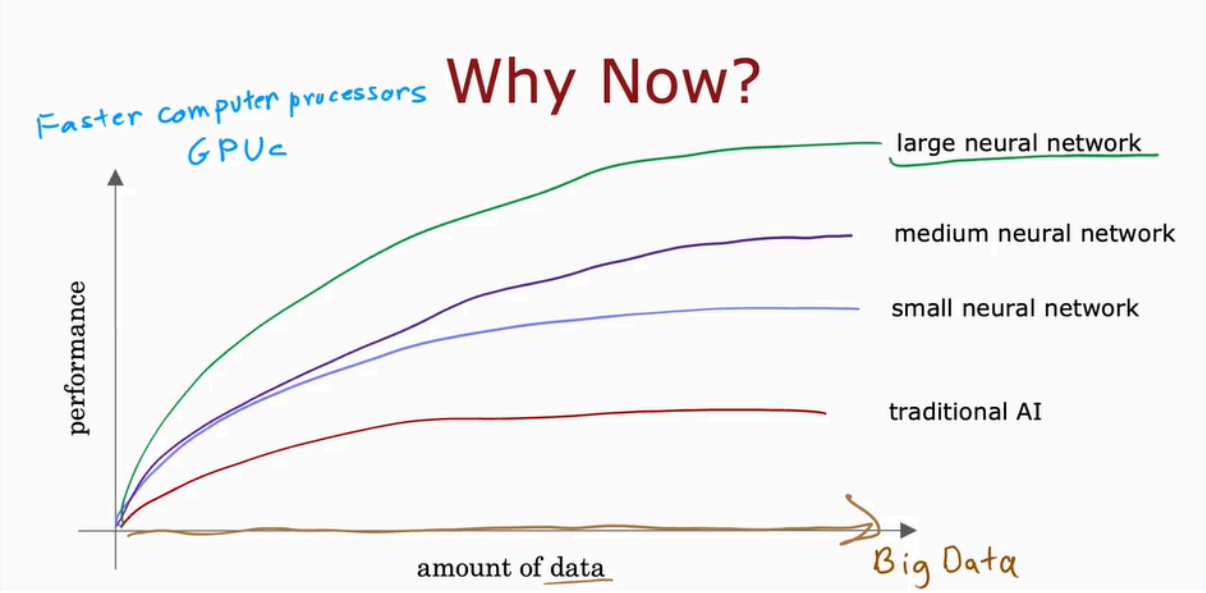
Here's a diagram illustrating what neurons in a brain look like. All of human thought is from neurons like this in your brain and mine, sending electrical impulses and sometimes forming new connections of other neurons.

- Given a neuron like this one(the top right neural), it has a number of inputs where it receives electrical impulses from other neurons, and then this neuron that I've circled carries out some computations and will then send this outputs to other neurons by this electrical impulses, and this upper neuron's output in turn becomes the input to this neuron down below, which again aggregates inputs from multiple other neurons to then maybe send its own output, to yet other neurons, and this is the stuff of which human thought is made



So the artificial neural network uses a very simplified Mathematical model of what a biological neuron does. I'm going to draw a little circle here (the dark blue one) to denote a single neuron.

- What a neuron does is it takes some inputs, one or more inputs, which are just numbers. It does some computation and it outputs some other number, which then could be an input to a second neuron, shown here on the right. When you're building an artificial neural network or deep learning algorithm, rather than building one neuron at a time, you often want to simulate many such neurons at the same time. In this diagram, I'm drawing three neurons. What these neurons do collectively is input a few numbers, carry out some computation, and output some other numbers.



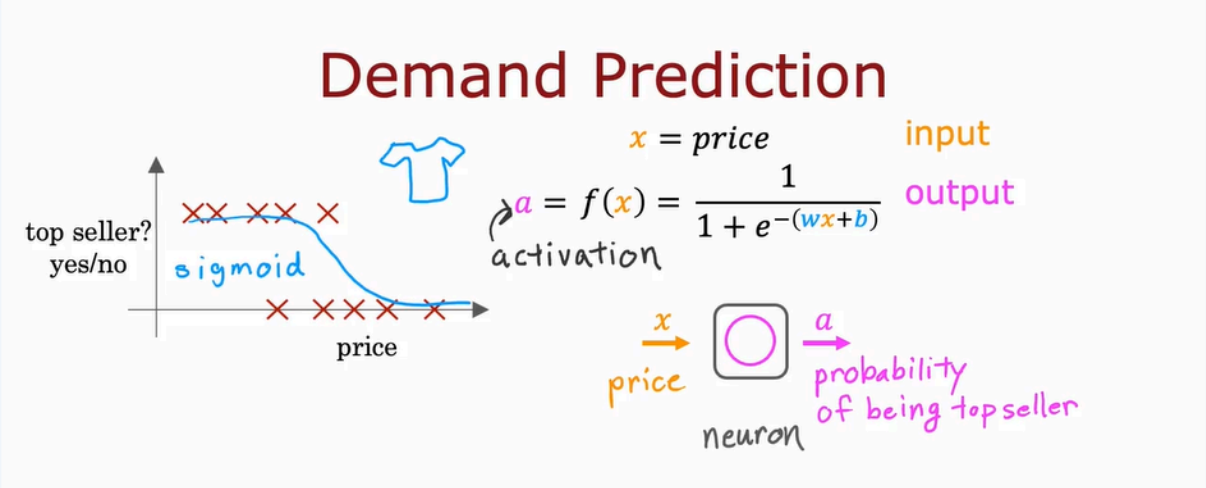
- Now, at this point, I'd like to give one big caveat, which is that even though I made a loose analogy between biological neurons and artificial neurons, I think that today we have almost no idea how the human brain works.
- In fact, every few years, neuroscientists make some fundamental breakthrough about how the brain works. I think we'll continue to do so for the foreseeable future. That to me is a sign that there are many breakthroughs that are yet to be discovered about how the brain actually works, and thus attempts to blindly mimic what we know of the human brain today, which is frankly very little, probably won't get us that far toward building raw intelligence.
- Certainly not with our current level of knowledge in neuroscience. Having said that, even with these extremely simplified models of a neuron, which

we'll talk about, we'll be able to build really powerful deep learning algorithms. So as you go deeper into neural networks and into deep learning, even though the origins were biologically motivated, don't take the biological motivation too seriously. In fact, those of us that do research in deep learning have shifted away from looking to biological motivation that much. But instead, they're just using engineering principles to figure out how to build algorithms that are more effective. But I think it might still be fun to speculate and think about how biological neurons work every now and then.

- The ideas of neural networks have been around for many decades. A few people have asked me, "Hey Andrew, why now? Why is it that only in the last handful of years that neural networks have really taken off?" This is a picture I draw for them when I'm asked that question and that maybe you could draw for others as well if they ask you that question. Let me plot on the horizontal axis the amount of data you have for a problem, and on the vertical axis, the performance or the accuracy of a learning algorithm applied to that problem. Over the last couple of decades, with the rise of the Internet, the rise of mobile phones, the digitalization of our society, the amount of data we have for a lot of applications has steadily marched to the right. Lot of records that use P on paper, such as if you order something rather than it being on a piece of paper, there's much more likely to be a digital record.
- Your health record, if you see a doctor, is much more likely to be digital now compared to on pieces of paper. So in many application areas, the amount of digital data has exploded. What we saw was with traditional machine-learning algorithms, such as logistic regression and linear regression, even as you fed those algorithms more data, it was very difficult to get the performance to keep on going up. So it was as if the traditional learning algorithms like linear regression and logistic regression, they just weren't able to scale with the amount of data we could now feed it and they weren't able to take effective advantage of all this data we had for different applications. What AI researchers started to observe was that if you were to train a small neural network on this dataset, then the performance maybe looks like this (the blue one).
- If you were to train a medium-sized neural network, meaning one with more neurons in it, its performance may look like that. If you were to train a very large neural network, meaning one with a lot of these artificial neurons,

then for some applications the performance will just keep on going up. So this meant two things, it meant that for a certain class of applications where you do have a lot of data, sometimes you hear the term big data toss around, if you're able to train a very large neural network to take advantage of that huge amount of data you have, then you could attain performance on anything ranging from speech recognition, to image recognition, to natural language processing applications and many more, they just were not possible with earlier generations of learning algorithms. This caused deep learning algorithms to take off, and this too is why faster computer processors, including the rise of GPUs or graphics processor units. This is hardware originally designed to generate nice-looking computer graphics, but turned out to be really powerful for deep learning as well. That was also a major force in allowing deep learning algorithms to become what it is today. That's how neural networks got started, as well as why they took off so quickly in the last several years.

Demand Prediction

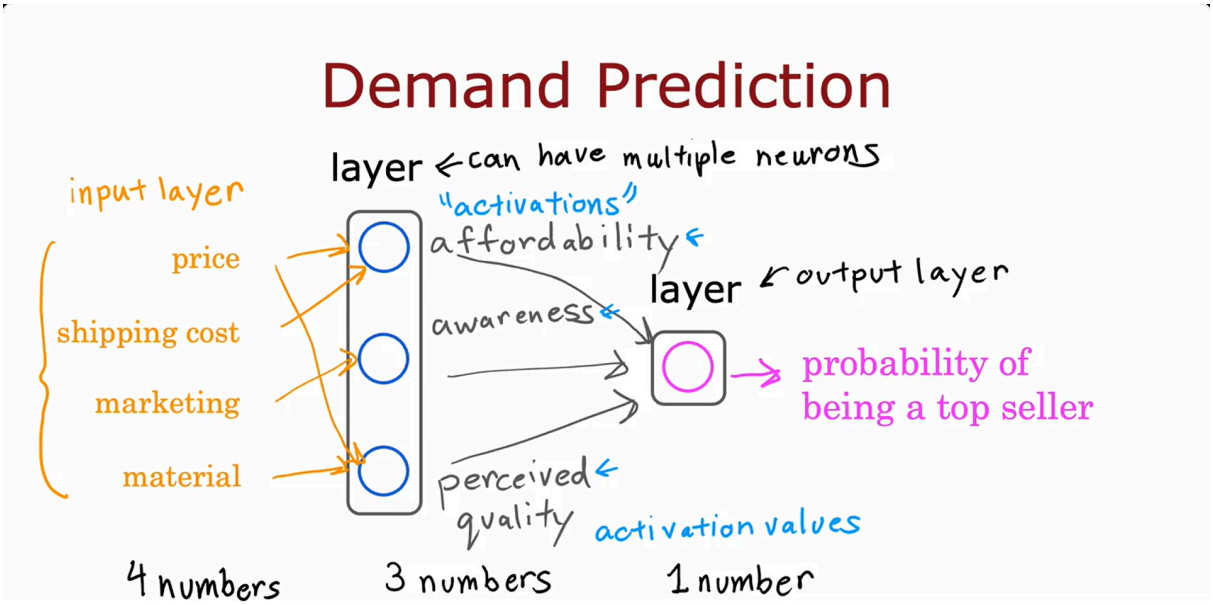


In this example, you're selling T-shirts and you would like to know if a particular T-shirt will be a top seller, yes or no, and you have collected data of different t-shirts that were sold at different prices, as well as which ones became a top seller. This type of application is used by retailers today in order to plan better inventory levels as well as marketing campaigns. If you know what's likely to be a top seller, you would plan, for example, to just purchase more of that stock in advance. In this example, the input feature x is the price of the T-shirt, and so that's the input to the learning algorithm. If you apply logistic regression to fit a

sigmoid function to the data that might look like that then the outputs of your prediction might look like this, $\frac{1}{1+e^{(-wx+b)}}$.

Previously, we had written this as f(x) as the output of the learning algorithm. In order to set us up to build a neural network, I'm going to switch the terminology a little bit and use the α to denote the output of this logistic regression algorithm. The term a stands for activation, and it's actually a term from neuroscience, and it refers to how much a neuron is sending a high output to other neurons downstream from it. It turns out that this logistic regression units or this little logistic regression algorithm, can be thought of as a very simplified model of a single neuron in the brain. Where what the neuron does is it takes us input the price x, and then it computes this formula on top, and it outputs the number a, which is computed by this formula, and it outputs the probability of this T-shirt being a top seller.

Another way to think of a neuron is as a tiny little computer whose only job is to input one number or a few numbers, such as a price, and then to output one number or maybe a few other numbers which in this case is the probability of the T-shirt being a top seller. As I alluded in the previous video, a logistic regression algorithm is much simpler than what any biological neuron in your brain or mine does. Which is why the artificial neural network is such a vastly oversimplified model of the human brain. Even though in practice, as you know, deep learning algorithms do work very well. Given this description of a single neuron, building a neural network now it just requires taking a bunch of these neurons and wiring them together or putting them together.



Let's now look at a more complex example of demand prediction. In this example, we're going to have four features to predict whether or not a T-shirt is a top seller. The features are:

- the price of the T-shirt,
- the shipping costs,
- the amounts of marketing of that particular T-shirt,
- as well as the material quality, is this a high-quality, thick cotton versus maybe a lower quality material?

Now, you might suspect that whether or not a T-shirt becomes a top seller actually depends on a few factors.

- First, one is the affordability of this T-shirt.
- Second is, what's the degree of awareness of this T-shirt that potential buyers have?
- Third is perceived quality to bias or potential bias saying this is a high-quality T-shirt.

What I'm going to do is create one artificial neuron to try to estimate the probability that this T-shirt is perceive as highly affordable. Affordability is mainly a function of price and shipping costs because the total amount of the pay is some of the price plus the shipping costs. We're going to use a little neuron here (The first blue circle), a logistic regression unit to input price and shipping costs and predict do people think this is affordable?

Second, I'm going to create another artificial neuron here to estimate, is there high awareness of this? Awareness in this case is mainly a function of the marketing of the T-shirt. Finally, going to create another neuron to estimate do people perceive this to be of high quality, and that may mainly be a function of the price of the T-shirt and of the material quality.

Price is a factor here because fortunately or unfortunately, if there's a very high priced T-shirt, people will sometimes perceive that to be of high quality because it is very expensive than maybe people think it's going to be of high-quality. Given these estimates of affordability, awareness, and perceived quality we then wire the outputs of these three neurons to another neuron here on the right, that then there's another logistic regression unit. That finally inputs those three numbers and outputs the probability of this t-shirt being a top seller.

In the terminology of neural networks, we're going to group these three neurons together into what's called a layer.

A layer is a grouping of neurons which takes as input the same or similar features, and that in turn outputs a few numbers together. These three neurons on the left form one layer which is why I drew them on top of each other, and this single neuron on the right (the pink one) is also one layer. The layer on the left has three neurons, so a layer can have multiple neurons or it can also have a single neuron as in the case of this layer on the right. This layer on the right is also called the output layer because the outputs of this final neuron is the output probability predicted by the neural network. In the terminology of neural networks we're also going to call affordability awareness and perceive quality to be activations.

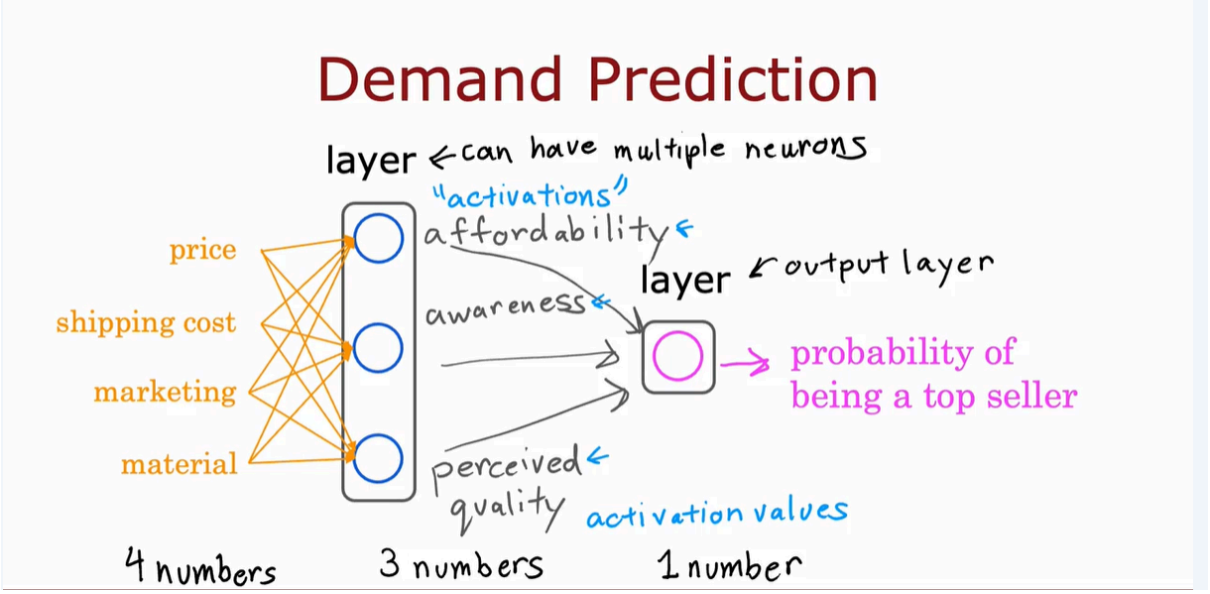
The term activations comes from biological neurons, and it refers to the degree that the biological neuron is sending a high output value or sending many electrical impulses to other neurons to the downstream from it. These numbers on affordability, awareness, and perceived quality are the activations of these three neurons in this layer, and also this output probability is the activation of this neuron shown here on the right. This particular neural network therefore carries out computations as follows.

It inputs four numbers then this layer (the layers of three neurons) of the neural network uses those four numbers to compute the new numbers also called activation values.

Then the final layer, the output layer of the neural network used those three numbers to compute one number. In a neural network this list of four numbers is also called the input layer, and that's just a list of four numbers.

Now, there's one simplification I'd like make to this neural network. The way I've described it so far, we had to go through the neurons one at a time and decide what inputs it would take from the previous layer.

For example, we said affordability is a function of just price and shipping costs and awareness is a function of just marketing and so on, but if you're building a large neural network it'd be a lot of work to go through and manually decide which neurons should take which features as inputs. The way a neural network is implemented in practice each neuron in a certain layer; say this layer in the middle, will have access to every feature, to every value from the previous layer, from the input layer, which is why I'm now drawing arrows from every input feature to every one of these neurons shown here in the middle.

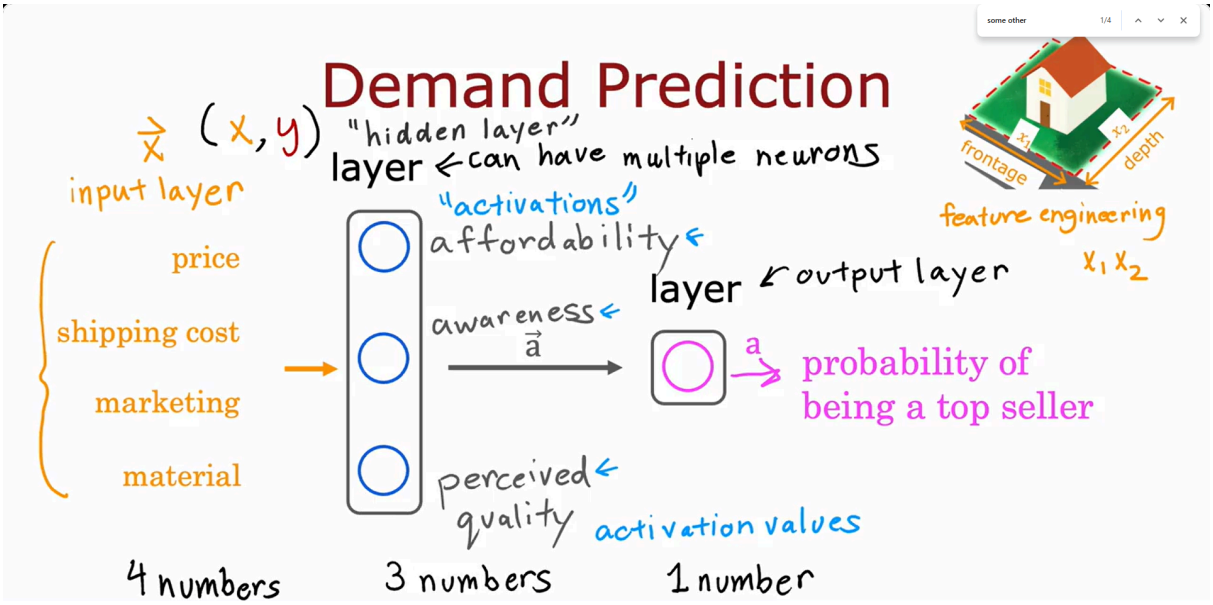


To further simplify the notation and the description of this neural network I'm going to take these four input features and write them as a vector x, and we're going to view the neural network as having four features that comprise this feature vector x.

This feature vector is fed to this layer in the middle which then computes three activation values. That is these numbers and these three activation values in turn becomes another vector which is fed to this final output layer that finally outputs the probability of this t-shirt to being a top seller. That's all a neural network is. It has a few layers where each layer inputs a vector and outputs another vector of numbers.

For example, this layer in the middle (the layer with three neurons) inputs four numbers x and outputs three numbers corresponding to affordability, awareness, and perceived quality. To add a little bit more terminology, you've seen that this layer is called the output layer and this layer is called the input layer. To give the layer in the middle a name as well, this layer in the middle is called a hidden layer.

I know that this is maybe not the best or the most intuitive name but that terminology comes from that's when you have a training set. In a training set, you get to observe both x and y. Your data set tells you what is x and what is y, and so you get data that tells you what are the correct inputs and the correct outputs. But your dataset doesn't tell you what are the correct values for affordability, awareness, and perceived quality. The correct values for those are hidden. You don't see them in the training set, which is why this layer in the middle is called a hidden layer.



I'd like to share with you another way of thinking about neural networks that I've found useful for building my intuition about it.

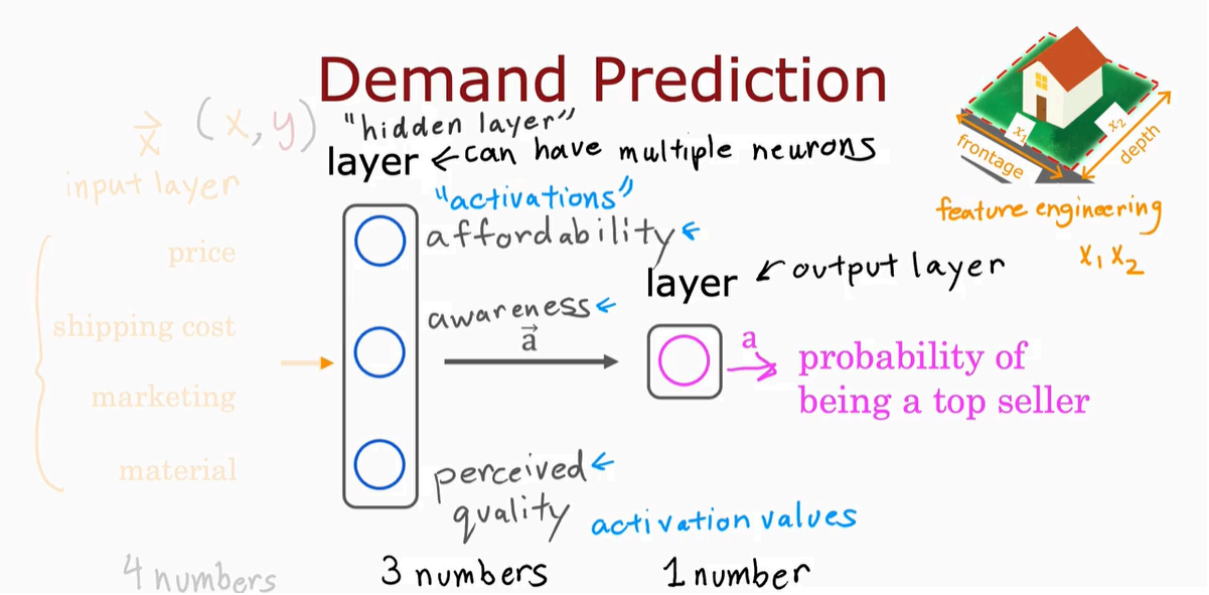
Just let me cover up the left half of this diagram (the image below), and see what we're left with. What you see here is that there is a logistic regression algorithm or logistic regression unit that is taking as input, affordability, awareness, and perceived quality of a t-shirt, and using these three features to estimate the probability of the t-shirt being a top seller.

This is just logistic regression. But the cool thing about this is rather than using the original features, price, shipping cost, marketing, and so on, is using maybe better set of features, affordability, awareness, and perceived quality, that are hopefully more predictive of whether or not this t-shirt will be a top seller.

One way to think of this neural network is, just logistic regression. But as a version of logistic regression, they can learn its own features that makes it easier to make accurate predictions. In fact, you might remember from the previous course, this housing example where we said that if you want to predict the price of the house, you might take the frontage or the width of lots and multiply that by the depth of a lot to construct a more complex feature, x_1 times x_2, which was the size of the lawn.

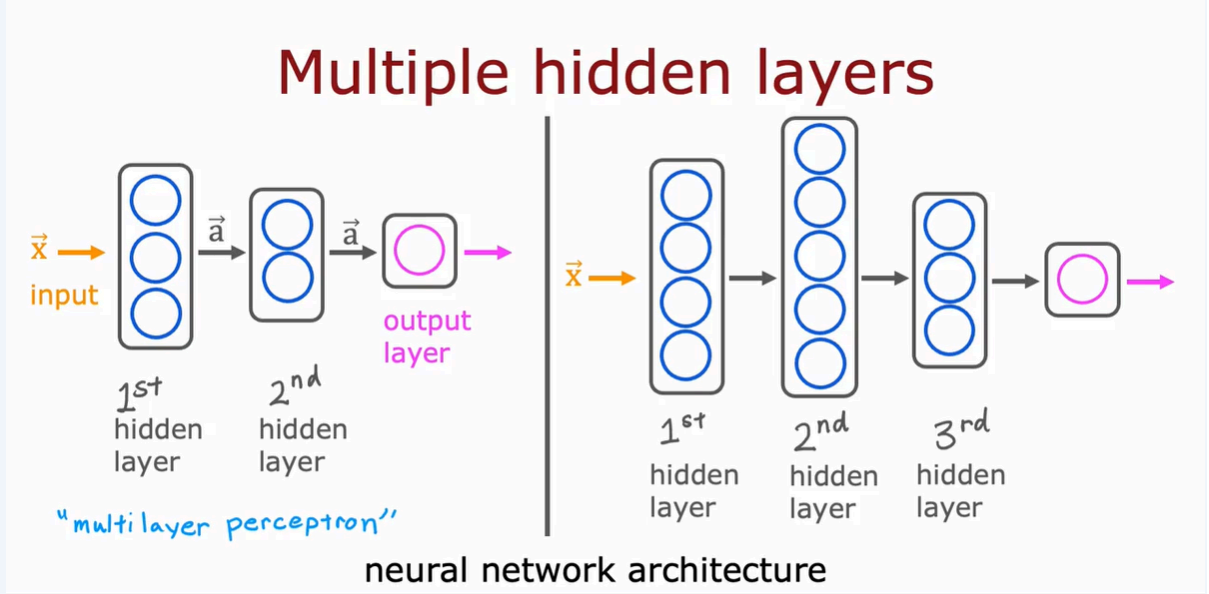
There we were doing manual feature engineering where we had to look at the features x_1 and x_2 and decide by hand how to combine them together to come up with better features. What the neural network does is instead of you needing to manually engineer the features, it can learn, as you'll see later, its own features to make the learning problem easier for itself. This is what makes

neural networks one of the most powerful learning algorithms in the world today.



To summarize, a neural network, does this, the input layer has a vector of features, four numbers in this example, it is input to the hidden layer, which outputs three numbers. I'm going to use a vector to denote this vector of activations that this hidden layer outputs. Then the output layer takes its input to three numbers and outputs one number, which would be the final activation, or the final prediction of the neural network.

One note, even though I previously described this neural network as computing affordability, awareness, and perceived quality, one of the really nice properties of a neural network is when you train it from data, you don't need to go in to explicitly decide what other features, such as affordability and so on, that the neural network should compute instead or figure out all by itself what are the features it wants to use in this hidden layer. That's what makes it such a powerful learning algorithm. You've seen here one example of a neural network and this neural network has a single layer that is a hidden layer.



neural network architecture

Let's take a look at some other examples of neural networks, specifically, examples with more than one hidden layer.

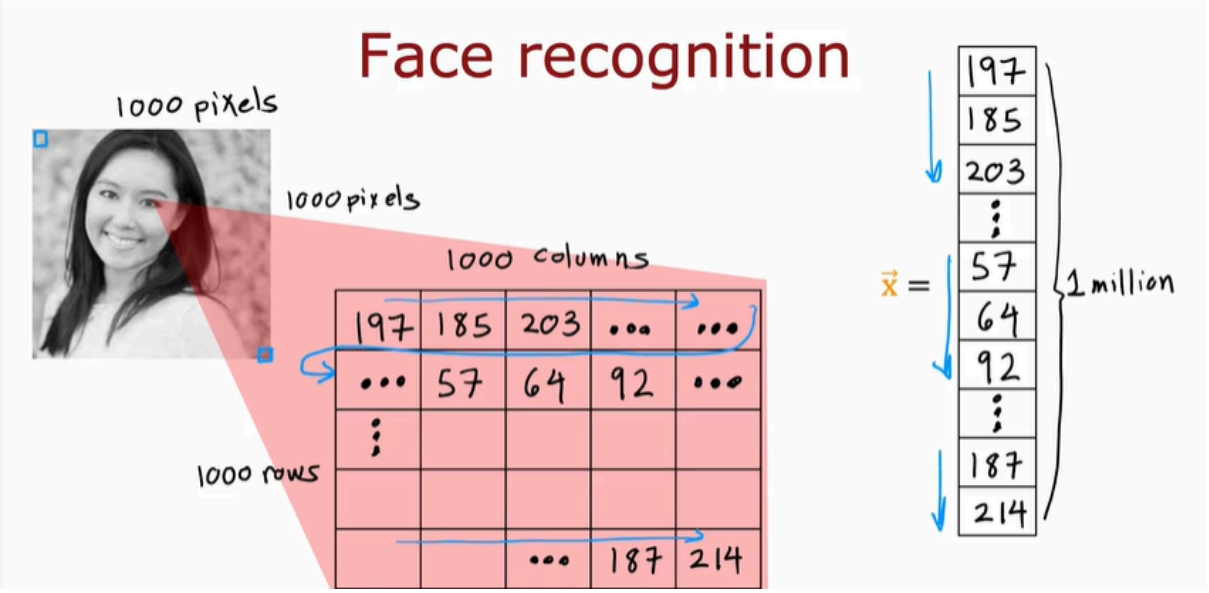
Here's an example (the architecture in the left). This neural network has an input feature vector X that is fed to one hidden layer. I'm going to call this the first hidden layer. If this hidden layer has three neurons, it will then output a vector of three activation values. These three numbers can then be input to the second hidden layer. If the second hidden layer has two neurons to logistic units, then this second hidden there will output another vector of now two activation values that maybe goes to the output layer that then outputs the neural network's final prediction.

Here's another example (the architecture in the right). Here's a neural network that it's input goes to the first hidden layer, the output of the first hidden layer goes to the second hidden layer, goes to the third hidden layer, and then finally to the output layer. When you're building your own neural network, one of the decisions you need to make is how many hidden layers do you want and how many neurons do you want each hidden layer to have. This question of how many hidden layers and how many neurons per hidden layer is a question of the architecture of the neural network. You'll learn later in this course some tips for choosing an appropriate architecture for a neural network.

But choosing the right number of hidden layers and number of hidden units per layer can have an impact on the performance of a learning algorithm as well. Later in this course, you'll learn how to choose a good architecture for your neural network as well. By the way, in some of the literature, you see this type of neural network with multiple layers like this called a multilayer perceptron. If

you see that, that just refers to a neural network that looks like what you're seeing here on the slide.

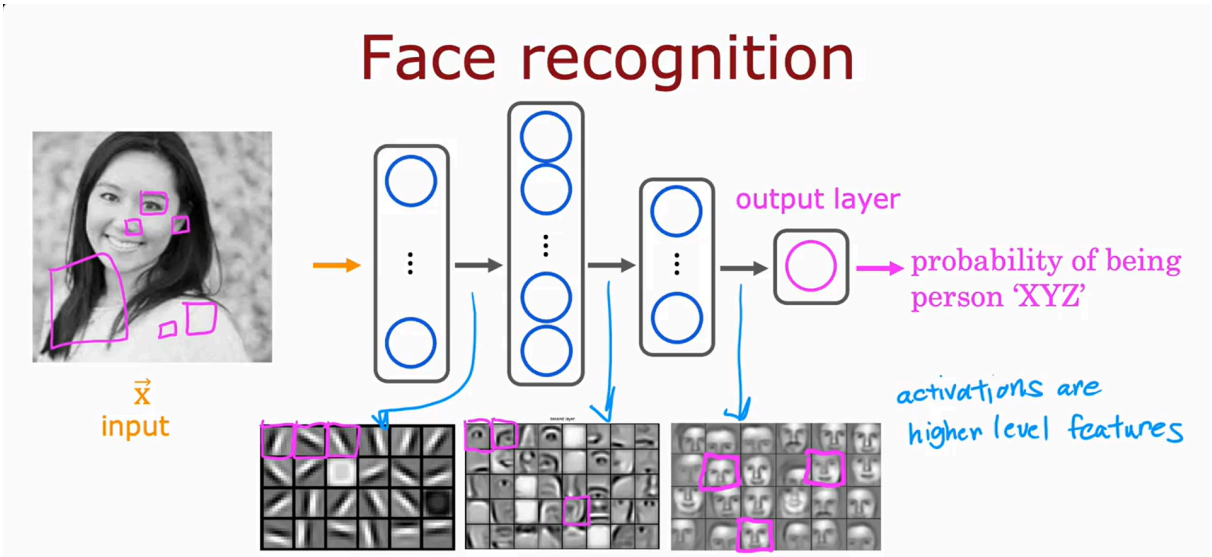
Example: Recognizing Images



Let's dive in. If you're building a face recognition application, you might want to train a neural network that takes as input a picture like this and outputs the identity of the person in the picture. This image is 1,000 by 1,000 pixels. Its representation in the computer is actually as 1,000 by 1,000 grid, or also called 1,000 by 1,000 matrix of pixel intensity values.

In this example, my pixel intensity values or pixel brightness values, goes from 0-255 and so 197 here would be the brightness of the pixel in the very upper left of the image, 185 is brightness of the pixel, one pixel over, and so on down to 214 would be the lower right corner of this image.

If you were to take these pixel intensity values and unroll them into a vector, you end up with a list or a vector of a million pixel intensity values. One million because 1,000 by 1,000 square gives you a million numbers. The face recognition problem is, can you train a neural network that takes as input a feature vector with a million pixel brightness values and outputs the identity of the person in the picture.



This is how you might build a neural network to carry out this task. The input image X is fed to this layer of neurons. This is the first hidden layer, which then extract some features. The output of this first hidden layer is fed to a second hidden layer and that output is fed to a third layer and then finally to the output layer, which then estimates, say the probability of this being a particular person.

One interesting thing would be if you look at a neural network that's been trained on a lot of images of faces and to try to visualize what are these hidden layers, trying to compute. It turns out that when you train a system like this on a lot of pictures of faces and you peer at the different neurons in the hidden layers to figure out what they may be computing this is what you might find.

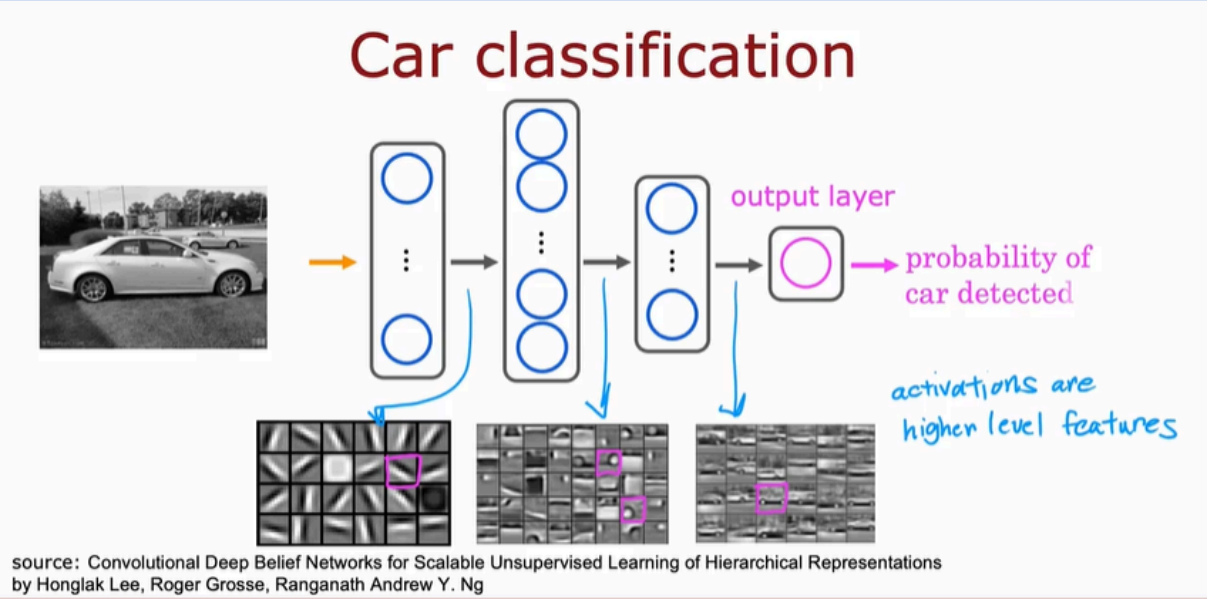
In the first hidden layer, you might find one neuron that is looking for the low vertical line or a vertical edge like that. A second neuron looking for a oriented line or oriented edge like that. The third neuron looking for a line at that orientation, and so on.

In the earliest layers of a neural network, you might find that the neurons are looking for very short lines or very short edges in the image. If you look at the next hidden layer, you find that these neurons might learn to group together lots of little short lines and little short edge segments in order to look for parts of faces.

For example, each of these little square boxes is a visualization of what that neuron is trying to detect.

- This first neuron looks like it's trying to detect the presence or absence of an eye in a certain position of the image.
- The second neuron, looks like it's trying to detect like a corner of a nose and maybe this neuron over here is trying to detect the bottom of an ear. Then as you look at the next hidden layer in this example, the neural network is aggregating different parts of faces to then try to detect presence or absence of larger, coarser face shapes.
- Then finally, detecting how much the face corresponds to different face shapes creates a rich set of features that then helps the output layer try to determine the identity of the person picture. A remarkable thing about the neural network is you can learn these feature detectors at the different hidden layers all by itself.

In this example, no one ever told it to look for short little edges in the first layer, and eyes and noses and face parts in the second layer and then more complete face shapes at the third layer. The neural network is able to figure out these things all by itself from data. Just one note, in this visualization, the neurons in the first hidden layer are shown looking at relatively small windows to look for these edges. In the second hidden layer is looking at bigger window, and the third hidden layer is looking at even bigger window. These little neurons visualizations actually correspond to differently sized regions in the image.



source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng

Just for fun, let's see what happens if you were to train this neural network on a different dataset, say on lots of pictures of cars, picture on the side. The same learning algorithm is asked to detect cars, will then learn edges in the

first layer. Pretty similar but then they'll learn to detect parts of cars in the second hidden layer and then more complete car shapes in the third hidden layer.

Just by feeding it different data, the neural network automatically learns to detect very different features so as to try to make the predictions of car detection or person recognition or whether there's a particular given task that is trained on. That's how a neural network works for computer vision application.